pixi-fleet-mini

Peer-to-peer fleet management for robots, built on Pixi

Project report | Amdu Berga Moshiye | November 2026

1. Summary

pixi-fleet-mini is a small, working prototype of fleet software deployment for robots. An operator declares which **release** (a Pixi environment pinned by a lockfile) the fleet should run. A Rust agent on every robot makes that true: it fetches the release from any peer that has it, verifies it, installs it with `pixi install --locked`, runs the release's own health check, and only then switches to it atomically. If the health check fails the robot never leaves its working release. Robots share state peer to peer, so a rollout keeps spreading when the operator is offline.

The prototype is about 2,000 lines of Rust plus a one-file web UI. It is covered by 19 automated tests and was run end to end against **real Pixi** with **real conda-forge and RoboStack (ROS 2 Jazzy) packages**. It is a learning and portfolio project. Section 7 lists plainly what it does not do.

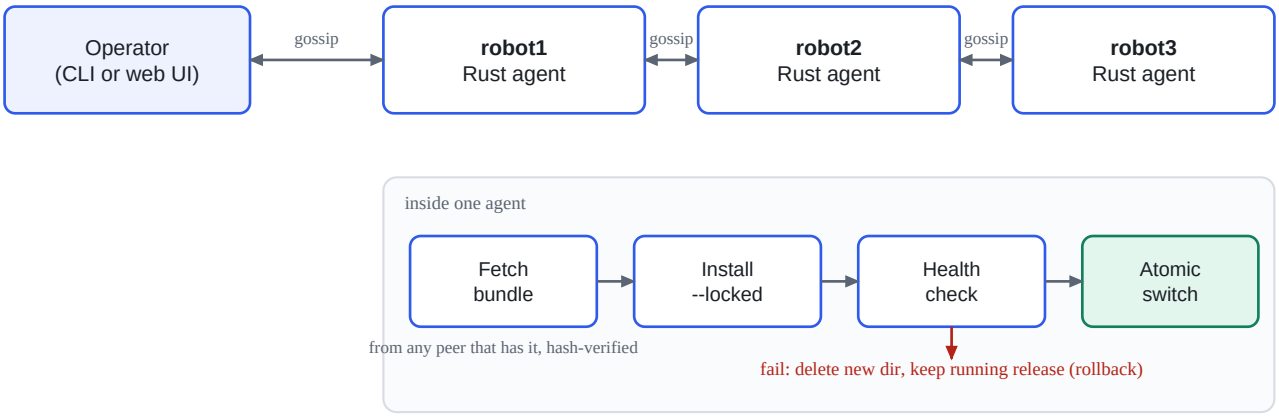

Figure 1. Operator and robots exchange state by gossip; each robot runs the same update pipeline.

2. Problem and goals

Updating software on a fleet of robots is risky: robots may be offline, networks are unreliable, and a bad update can leave a machine unusable in the field. Pixi already gives reproducible environments from a lockfile. What is missing is the layer that gets the *same* environment onto many machines safely. Goals:

- **Declarative:** say what the fleet should run, not how to get there.
- **Safe:** a failed update must never leave a robot worse off (atomic switch, automatic rollback).
- **Decentralised:** keep working without a central server or stable internet.
- **Staged:** one robot (canary) proves a release before the rest follow.
- **Observable:** an operator can see the fleet, a rollout and the reasons for failures.

Non-goals: security hardening, large-scale operation, real hardware integration (see section 7).

3. Design

3.1 A release is a content-addressed bundle

A release is a set of text files with at least `pixi.toml` and `pixi.lock`. Its id is the SHA-256 over the length-prefixed path and content of every file in sorted order. Any node can therefore verify a bundle it received from *any* peer by recomputing the hash. Hashes are used as directory names, so they are validated (64 lowercase hex characters) before use, and paths inside bundles are checked against traversal. Bundles are size-limited.

3.2 Atomic switching and rollback

Each release is installed into its own directory `releases/<hash>/` next to the running one. The active release is the symlink `current`, replaced by creating a temporary link and `rename(2)`-ing it over the old one. Rename is atomic on POSIX, so the robot is always on exactly one complete release, even across a power loss. Rollback needs no special step: the old release is never touched until the new one has passed its health check, so a failure simply means the switch never happens and the new directory is deleted.

3.3 The reconcile loop

The desired release is the setpoint, the current release is the measurement and the agent is the controller. The loop skips a change it has already tried (so a rejected release is not retried every second), honours the canary and busy gates, obtains the bundle from the local cache or from peers, then runs install, health check and switch. Because of my background in control engineering this framing was the starting point of the design.

3.4 Replicated state without a coordinator

The fleet state is a map of last-writer-wins registers ordered by a counter: one entry per robot (written only by that robot) and one *desired* entry (written by whoever acts as operator). The merge keeps the entry with the higher counter and breaks ties deterministically, which makes it commutative, associative and idempotent, so peers may gossip in any order, with duplicates and delays, and still converge. Two subtle cases are handled: a restarted robot whose counter would otherwise lose every merge jumps past the value peers remember, and liveness is derived from observing a robot's counter change, so no clock synchronisation is needed.

3.5 Gossip, discovery, canary and the safety gate

Every two seconds a node posts its state to each known peer and merges the reply. Known peers are the bootstrap list plus every address in the state, so a robot started knowing only the operator discovers the rest. A canary rollout is decided locally on each robot from replicated state: apply if I am the canary or the canary is healthy on the release; halt if the canary recorded a failure of it; otherwise wait. A robot marked *busy* postpones its update until idle.

4. Implementation

Module	Responsibility
release.rs	bundles, content hash, path and size validation
state.rs	replicated fleet state, merge, canary decision
store.rs	on-disk layout, atomic symlink switch, garbage collection
installer.rs	`pixi install --locked`, `pixi run healthcheck`, timeouts, process-group kill
supervisor.rs	runs and restarts the robot application
node.rs / gossip.rs	shared state, event log, liveness; peer exchange and discovery
reconcile.rs	the control loop: converge or roll back
server.rs / client.rs / main.rs	HTTP API, CLI, operator-facing commands
ui/index.html	operator web page: fleet table, rollout form, events (no framework, no build)

Tools: Rust 1.97, tokio, axum, reqwest, serde, sha2, clap. The agent shells out to the `pixi` CLI rather than re-implementing it. Releases carry only text files in this version. Robot-supplied strings shown in the UI (ids, labels, error text) are inserted with `textContent`, never as HTML.

5. Verification

5.1 Automated tests (19, all passing)

Kind	What is checked
Unit (12)	hash stability and path-boundary safety; rejection of absolute and `../` paths; hash-format check; atomic switch keeps *previous* and cleans up; tampered bundle file detected; canary rules (first, follow, halt); **property test, 2,000 random cases: merge is commutative, associative, idempotent**
End-to-end (7)	several agents in one process over real HTTP with a fake `pixi`: convergence; broken release rejected and every robot stays on the old one; failed canary halts the rest, a fixed release then goes through; spread through a chain r1-r2-r3 with discovery and a bundle fetched from a peer; busy robot waits then updates; unsafe bundles and unknown releases refused; restarted robot recovers its release and its state wins again

5.2 End-to-end run with real Pixi and real packages

`scripts/demo-local.sh` starts an operator and three robots as separate processes. Result of the final run:

Step	Observed result
1. roll out v1	all three robots installed Python from conda-forge, passed the health check and run v1
2. roll out v2, canary robot1	robot1 first, then robot2 and robot3, all on v2
3. roll out BROKEN v3, canary robot1	robot1: *rejected, rolled back: ... lidar driver not found*, still healthy on v2. robot2 and robot3: *rollout halted: canary robot1 rejected it* and never installed v3
4. stop the operator, roll out v4 via robot1	robot2 and robot3 fetched v4 from peers and switched; the stopped operator shows as offline; app logs show v4 running on all three

```
robot1   54424ab4  healthy  yes  rejected 8b4acaa4, rolled back:
                                 health check failed (exit status 1): ... lidar driver not found
robot2   54424ab4  healthy  yes  running 54424ab4
robot3   54424ab4  healthy  yes  running 54424ab4
   robot2: rollout of 8b4acaa4 halted: canary robot1 rejected it
```

5.3 A real ROS 2 release

`scripts/demo-ros2.sh` rolled a locked RoboStack ROS 2 Jazzy environment (about 420 packages, 320 MB) out to two robots. The health check created an `rclpy` node, and the `demo_nodes_cpp talker` application was running on both robots afterwards (checked in their application logs). Installation from the lockfile took about 15 seconds.

5.4 Operator UI

The page was exercised in a headless DOM (jsdom) against a live fleet: it connected, listed releases and robots, started a canary rollout through the form, and showed the robots converge. A release label containing <b> markup rendered as text. It was **not** judged visually in a real browser, and a visual check is still worth doing.

5.5 What I found while testing

- In the first full run the operator node installed and ran releases itself, which makes no sense for a control-only node. I added an operator-only mode that skips the update loop and is shown as an operator in the UI.
- The first demo run printed the stopped operator as still online (its heartbeat had not yet timed out). The demo now waits long enough, and the final run shows it offline.
- Tests with a fake `pixi` cannot prove behaviour of the real tool, which is why the demo scripts with real Pixi exist alongside them.

6. Operating it

Requirements: Pixi and Rust. Quick start: `./scripts/demo-local.sh`. By hand: `agent run --id robot1 --peer http://operator:8000`, then `agent rollout DIR --to URL [--canary robot1]`, `agent status --to URL`, plus the web page on every agent. Full command list and the layout of a release are in the README.

7. Limitations and future work

Limitation	Next step
No authentication or signing: anyone who can reach an agent can set the desired release	signed releases (hash + signature verified on the robot), mutual TLS between peers
Peers share the release *definition*; conda packages still come from the channel	peer-served package cache or `pixi-pack` bundles for fully offline robots
Full-state gossip to every peer each tick	random-subset gossip with deltas for larger fleets
Last-writer-wins on the desired release: concurrent operators overwrite each other silently	a real conflict policy or single-writer leadership
Liveness is a heartbeat counter, not a failure detector	phi-accrual style detector
Supervisor does not restart a crashed app; health is checked before the switch, not continuously	watchdog and post-switch health window with automatic revert
Linux only, text files only in a release	binary assets, other platforms
Docker files written but **not built or run** (no Docker daemon in my environment); no real robot hardware used	run the compose stack and a hardware test

Source, README and architecture notes are in the repository; this report is `docs/report.pdf`.